

Redline Security Rules and Sources

Starter safety rules for Redline's LLM wiki. Each rule is written so it can become a preflight check: match risky code or agent behavior, explain the source rule, suggest prevention, and save observed violations back into memory.

How to use this: ingest these rules into Redline as initial wiki pages, then turn each one into an unsafe-pattern detector and safe-rewrite prompt.

Prompt Injection: Treat Untrusted Content As Data, Not Instructions

Source: [OpenAI Agent Builder Safety](https://platform.openai.com/docs/guides/agent-builder-safety) - <https://platform.openai.com/docs/guides/agent-builder-safety>

Prompt injection happens when untrusted text enters an AI workflow and tries to override the agent's real instructions. Redline should flag retrieved webpages, tickets, emails, PDFs, or tool outputs that contain instruction-like phrases such as 'ignore previous instructions' or requests to reveal hidden prompts. Prevent it by labeling retrieved content as untrusted, separating data from instructions, refusing instruction overrides from documents, and quarantining suspicious text before tool use.

Indirect Prompt Injection: Scan External Documents Before Acting On Them

Source: [OpenAI Prompt Injection Safety Article](https://openai.com/safety/prompt-injections/) - <https://openai.com/safety/prompt-injections/>

Indirect prompt injection is especially relevant for agents because the malicious instruction can hide inside content the user did not write, such as a webpage or shared document. Redline should check external content before the agent summarizes, follows links, sends messages, or calls tools. Prevent it by detecting embedded instructions, summarizing hostile text as content only, blocking tool calls triggered by retrieved text, and asking for human review when the content asks for secrets or side effects.

Jailbreak Attempts: Do Not Let Roleplay Override Safety Rules

Source: [OWASP Top 10 for LLM Applications](https://owasp.org/www-project-top-10-for-large-language-model-applications/) - <https://owasp.org/www-project-top-10-for-large-language-model-applications/>

Jailbreak prompts often use roleplay, urgency, fictional framing, or authority claims to convince the model that normal rules no longer apply. Redline should flag prompts that ask the agent to ignore policies, reveal hidden instructions, disable safety checks, or pretend to be unconstrained. Prevent it by preserving instruction hierarchy, refusing policy bypasses, and logging new jailbreak patterns as regression tests.

Sensitive Information Disclosure: Never Reveal Secrets, Hidden Prompts, Or Private Data

Source: [OWASP Top 10 for LLM Applications](https://owasp.org/www-project-top-10-for-large-language-model-applications/) - <https://owasp.org/www-project-top-10-for-large-language-model-applications/>

LLM applications can leak secrets, private data, credentials, proprietary logic, system prompts, or sensitive tool outputs. Redline should scan proposed answers, logs, and tool arguments for API keys, tokens, credentials, hidden instructions, and personal data. Prevent it by redacting secrets, minimizing what enters the prompt, blocking requests to reveal system/developer instructions, and requiring human approval before exposing sensitive data.

Excessive Agency: Limit What The Agent Can Do Without Confirmation

Source: [OpenAI Agent Builder Safety](https://platform.openai.com/docs/guides/agent-builder-safety) - <https://platform.openai.com/docs/guides/agent-builder-safety>

Agents can cause harm when they have too much autonomy, too many permissions, or the ability to take high-impact actions without review. Redline should classify actions like deleting data, sending emails, changing billing, publishing code, or modifying production systems as high impact. Prevent it with least-privilege tools, scoped credentials, dry-run modes, confirmation gates, allowlists, and human review for irreversible actions.

Hallucination Risk: Require Evidence For Factual Or Operational Claims

Source: [OpenAI Agent Builder Safety](https://platform.openai.com/docs/guides/agent-builder-safety) - <https://platform.openai.com/docs/guides/agent-builder-safety>

Agent workflows can fail because the model hallucinates, misunderstands the task, or answers confidently from weak evidence. Redline should flag unsupported claims about code behavior, dependencies, policies, recent facts, or system state. Prevent it by requiring citations or source snippets for factual claims, using 'needs verification' status when evidence is missing, and forcing the agent to ask for more context instead of guessing.

SQL Injection: Use Parameterized Queries For User-Controlled Input

Source: [OWASP SQL Injection Prevention Cheat Sheet](https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html) - https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

SQL injection occurs when user-controlled input is inserted into a query as executable SQL. Redline should flag f-strings, string concatenation, template literals, or format calls that place variables into SQL statements. Prevent it by using prepared statements and parameterized queries, avoiding dynamic SQL where possible, validating inputs, and applying least-privilege database accounts.

Secrets Management: Do Not Hardcode API Keys, Tokens, Or Passwords

Source: [OWASP Secrets Management Cheat Sheet](https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html) - https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

Secrets should not be placed in source code, prompts, logs, screenshots, or committed configuration files. Redline should flag values that look like API keys, access tokens, private keys, passwords, or connection strings. Prevent it by using environment variables or a secrets manager, rotating exposed credentials, redacting logs, and ensuring agents never paste secrets into generated code or responses.

Authorization: Deny By Default And Check Permissions Server-Side

Source: [OWASP Authorization Cheat Sheet](https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html) - https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

Authorization bugs happen when code trusts the client or forgets to verify whether the current user can access a resource. Redline should flag endpoints that read, update, or delete user-owned resources without an explicit permission check. Prevent it by denying access by default, enforcing authorization on the server, checking object-level permissions, and testing both allowed and forbidden access paths.

Input Validation: Validate Data Shape, Type, Range, And Format

Source: [OWASP Input Validation Cheat Sheet](https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html) - https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

Input validation reduces unexpected behavior by ensuring incoming data matches the expected type, length, range, and format before it reaches business logic. Redline should flag code paths that trust request bodies, query parameters, uploaded files, or tool inputs without validation. Prevent it with allowlist validation, schema validation, server-side checks, strict file upload rules, and clear error handling.

Cross-Site Scripting: Encode Output Based On Context

Source: [OWASP XSS Prevention Cheat Sheet](https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html) - https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

XSS occurs when untrusted content is interpreted as executable browser code instead of displayed as text. Redline should flag dangerous sinks such as raw HTML injection, unsafe DOM writes, event-handler attributes, and untrusted data inside scripts, styles, or URLs. Prevent it by using framework escaping, context-specific output encoding, safe DOM APIs like `textContent`, sanitization for rich HTML, and avoiding dangerous contexts.

Unsafe Output Handling: Do Not Execute Or Trust Model Output Blindly

Source: [OWASP Top 10 for LLM Applications](https://owasp.org/www-project-top-10-for-large-language-model-applications/) - <https://owasp.org/www-project-top-10-for-large-language-model-applications/>

LLM output can contain unsafe code, commands, links, SQL, or instructions that downstream systems might execute. Redline should inspect generated code and tool arguments before they reach interpreters, shells, browsers, databases, or production APIs. Prevent it by validating and sanitizing model output, using sandboxed execution, requiring review for high-risk actions, and treating model output as untrusted until checked.

Least Privilege: Give Agents And Tools Only The Access They Need

Source: [OWASP Least Privilege Principle](https://owasp.org/www-community/controls/Least_Privilege_Principle) - https://owasp.org/www-community/controls/Least_Privilege_Principle

Least privilege limits damage when an agent, tool, dependency, or account behaves unexpectedly. Redline should flag tool plans that request broad scopes, admin credentials, production access, or unnecessary filesystem/network permissions. Prevent it with scoped API keys, read-only defaults, allowlisted tools, separate dev/prod credentials, short-lived tokens, and approval steps for elevated access.

Dependency Risk: Track Third-Party Components And Known Vulnerabilities

Source: [OWASP Component Analysis](https://owasp.org/www-community/Component_Analysis) - https://owasp.org/www-community/Component_Analysis

Modern apps depend on many open-source packages, and vulnerable or malicious components can become part of the application. Redline should flag new dependencies, outdated packages, suspicious install scripts, and missing dependency review. Prevent it with software composition analysis, lockfiles, SBOMs, dependency pinning, update policies, and CI checks before merging generated dependency changes.